

โครงงาน

Hiwsaurus

(เกมหิวซอรัส)

จัดทำโดย

นายจุมพลภัทร์ สาเกกูล

รหัสนักศึกษา6604062630111 ตอนเรียนที่ 1

เสนอ

ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

สารบัญ

หัวข้อ	หน้า
- เกี่ยวกับโครงงาน	1
- บทที่ 1 บทนำ	1
- บทที่ 2 ส่วนการพัฒนา	7
- บทที่ 3 สรุป	25

เกี่ยวกับโครงงาน

ชื่อโปรเจค: Hiwsaurus (หิวซอรัส) เกมไดโนเสาร์นักล่าพันธุ์หายาก

นำเสนอโดย: นายจุมพลภัทร์ สาเกกูล รหัสนักศึกษา 6604062630111

อาจารย์ผู้สอน: ผู้ช่วยศาสตราจารย์ สถิตย์ ประสมพันธ์

Source Code: https://github.com/6604062630111/Hiwsaurus_OOP_6604062630111.git

บทที่ 1 บทนำ

ที่มาและความสำคัญ

โครงงานนี้จัดทำขึ้นเพื่อวัดผลความสามารถในการเรียนวิชา Object Oriented Programming โดยนำความรู้ที่เรียนมาสร้างเป็นชิ้นงานในรูปแบบเกม และใช้แนวคิดการเขียนโปรแกรมเชิงวัตถุ ทั้งการสร้างคลาส การสืบทอด การห่อหุ้มข้อมูล การใช้โพลิมอร์ฟิซึม และคลาสนามธรรม มาออกแบบโครงสร้างของเกมให้เป็นระบบและต่อยอดได้ง่าย

ประเภทของโครงการ

เกม (Game) 2 มิติ แนวเก็บของที่ตกลงมา (Catcher) เล่นคนเดียวบนหน้าจอคอมพิวเตอร์ พัฒนาด้วยภาษา Java

ประโยชน์

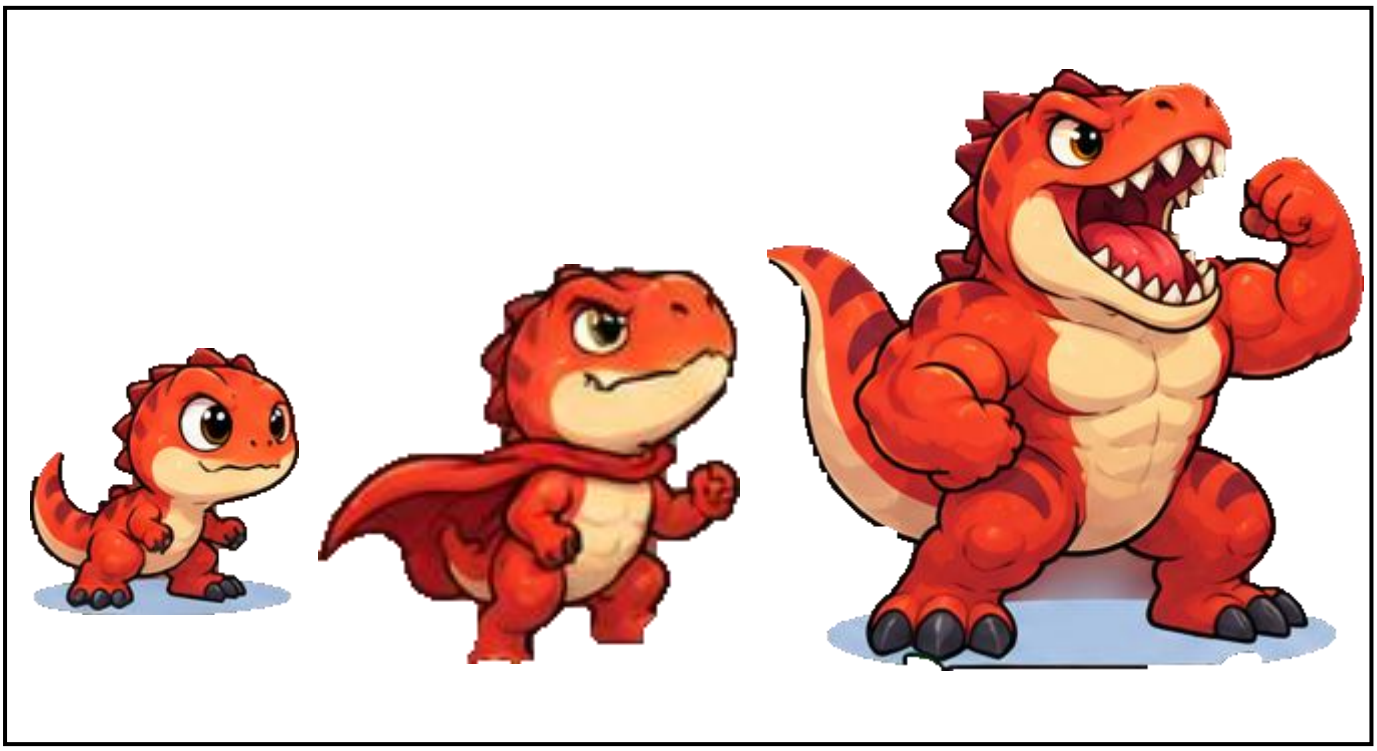
1. ได้ฝึกปฏิกิริยาการตอบสนองของผู้เล่น
2. ได้นำแนวคิดการเขียนโปรแกรมเชิงวัตถุมาประยุกต์ใช้กับงานจริง
3. ช่วยฝึกฝนไหวพริบและการตัดสินใจในเวลาสั้นของผู้เล่น
4. เพิ่มความสนุกสนาน ช่วยผ่อนคลาย และเพิ่มสมาธิ

ขอบเขตของโครงการ

1. เกม 2 มิติ บนหน้าจอคอมพิวเตอร์ เล่นได้คนเดียว
2. มี 3 บท (Chapter) บทละ 3 ด่าน (Level) รวม 9 ด่าน และด่านสุดท้ายของแต่ละบทเป็นด่านบอส
3. ผ่านด่านปกติด้วยการเก็บแต้มให้ถึงเป้าหมาย ส่วนด่านบอสต้องเก็บเนื้อเพื่อลดเลือดบอสให้หมด

Proposal

ตัวละคร

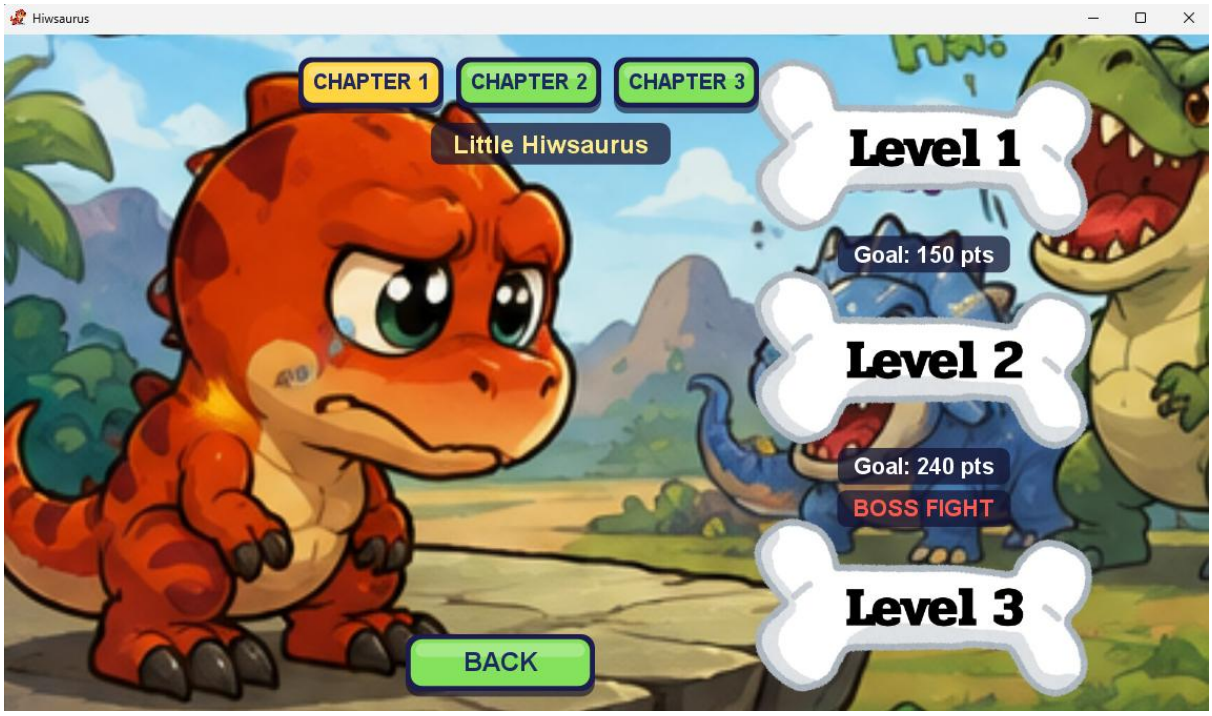


Story Board

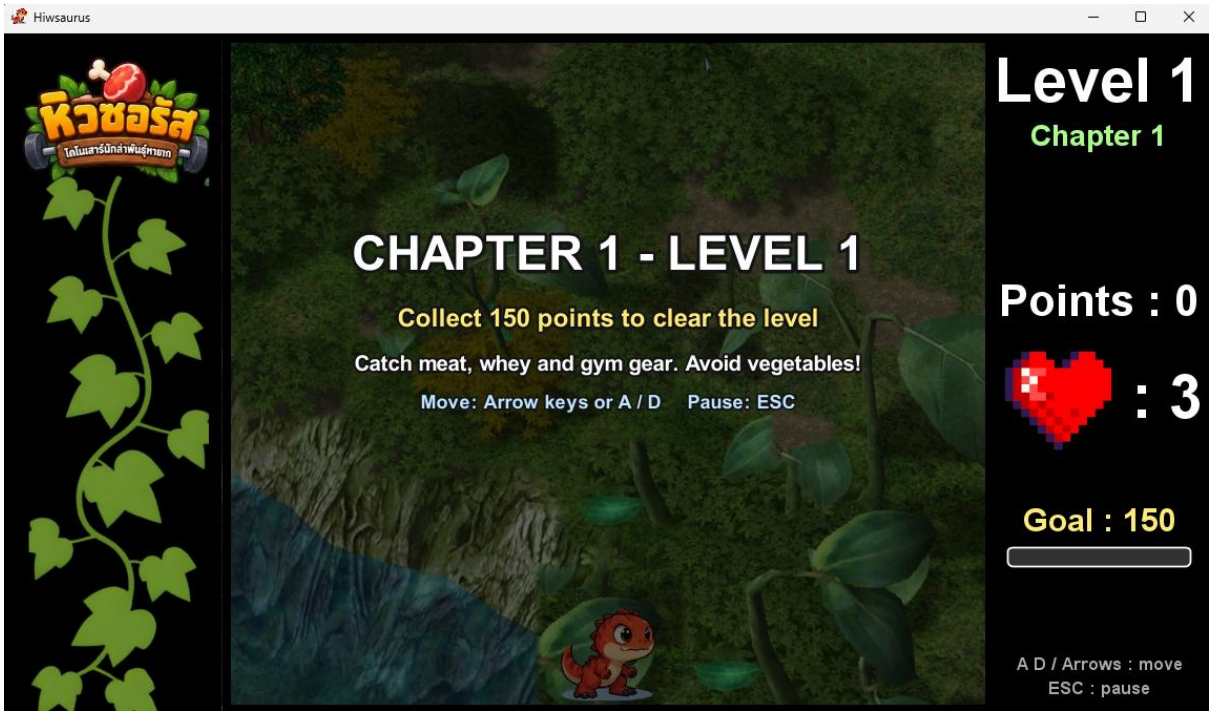
- หน้าเริ่มเกม



- หน้าเลือกด่าน



- หน้าตัวเกม (ด่าน)



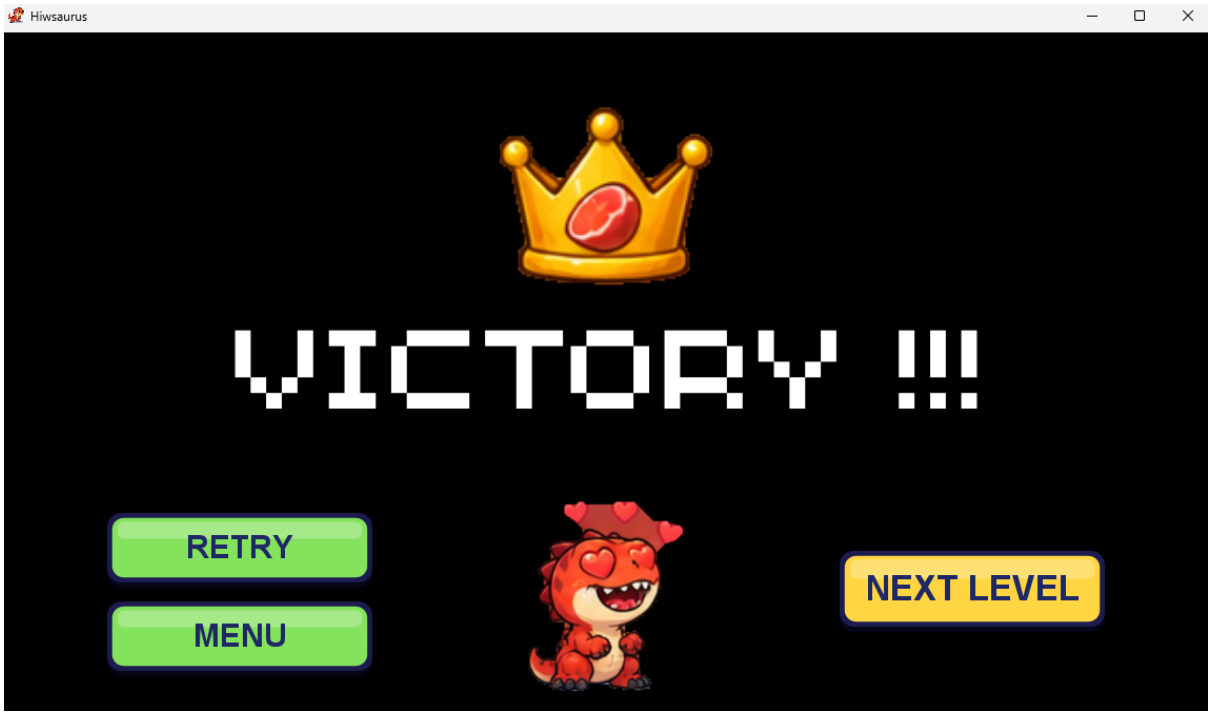
- เคลื่อนไหวหิวซอรัสเพื่อไปเก็บของที่ต้องการให้ถูก เมื่อเก็บของถูกจะได้แต้มเพิ่ม โดยของแต่ละอย่างให้แต้มต่างกัน



- เมื่อเก็บของไม่ถูกต้อง จะถูกหักพลังชีวิต



- เมื่อแต้มครบถึงจำนวนที่ด่านกำหนดจะผ่านด่านและสามารถไปด่านถัดไปได้



- เมื่อพลังชีวิตหมดจะ Game Over



ตารางแผนการทำงาน

ลำดับ	รายการ	ต้น-กลาง กันยายน	กลาง-ปลาย กันยายน	ตุลาคม
1	ค้นหาและสร้างองค์ประกอบตัวละคร/ฉาก	√		
2	หาข้อมูลที่เกี่ยวข้อง	√		
3	ออกแบบโปรแกรม		√	
4	ทำเอกสาร		√	
5	ตรวจสอบข้อผิดพลาด			√

บทที่ 2
การพัฒนา

รายละเอียดเกม

หิวซอรัส ไดโนเสาร์นักล่าพันธุ์หายากจากยุคดึกดำบรรพ์ ถูกเพื่อนๆ ในฝูงล้อเลียนว่ามีขนาดตัวเล็ก เขาจึงตั้งใจเข้า Dino Gym เพื่อออกกำลังกาย กินเนื้อและเวย์เพื่อสร้างกล้ามเนื้อ และสาบานว่าจะกลับไปแก้แค้นเพื่อนในฝูง ทุกบทของเกมจะจบด้วยการสู้กับไดโนเสาร์เพื่อนในฝูงที่เป็นบอส (สีส้ม สีฟ้า และสีเขียว ตามลำดับบทที่ 1 ถึง 3) ตัวละครจะโตขึ้นเรื่อยๆ จนกลายเป็นราชานักล่าแห่งยุคดึกดำบรรพ์

วิธีการเล่น

1. ใช้ปุ่มลูกศร ⟵ ⟶ หรือปุ่ม A D เพื่อขยับหิวซอรัสไปทางซ้ายและขวา ส่วนปุ่ม ESC หรือ P ใช้หยุดเกมชั่วคราว
2. ของจะตกลงมาจากด้านบน ให้ขยับตัวไปรับของที่กินได้ ได้แก่ เนื้อ เวย์โปรตีน และอุปกรณ์ออกกำลังกาย แต่ละอย่างให้แต้มไม่เท่ากัน
3. ถ้าเก็บผัก จะเสียพลังชีวิต 1 ดวง ตัวละครเปลี่ยนเป็นภาพเศร้าและกะพริบชั่วครู่ (ช่วงนี้ผักที่ตามมาจะไม่ทำให้เสียชีวิตซ้ำ) ถ้าพลังชีวิตหมดจะแพ้ (Game Over)
4. ด่านปกติ ผ่านด่านเมื่อเก็บแต้มได้ถึงเป้าหมาย ด่านบอสให้เก็บเนื้อ เวย์ หรืออุปกรณ์เพื่อลดเลือดบอสจนหมด โดยบอสจะเดินไปมาและปล่อยของลงมาจากตำแหน่งของตัวเอง
5. เมื่อชนะจะแสดงหน้า VICTORY พร้อมปุ่มไปด่านถัดไป เล่นซ้ำ หรือกลับเมนู เมื่อแพ้จะแสดงหน้า GAME OVER พร้อมปุ่มเล่นซ้ำหรือกลับเมนู

ของในเกมและคะแนนที่จะได้รับ

ประเภท	รายการ	ผลต่อผู้เล่น	ลดเลือดบอส
เนื้อ (Meat)	สเต็ก ขาไก่ ขาหมู ซี่โครง เนื้อย่าง ปลา เนื้อสไลซ์	+10 แต้ม	6
อุปกรณ์ (Equipment)	เคทเทิลเบล บาร์เบลเล็ก บาร์เบลใหญ่	+20 แต้ม	10
เวย์โปรตีน (Whey)	เชคเกอร์เวย์	+30 แต้ม	15
ผัก (Vegetable)	แครอท บรอกโคลี กะหล่ำปลี มะเขือเทศ ข้าวโพด ฟักทอง	เสียชีวิต 1 ดวง	-

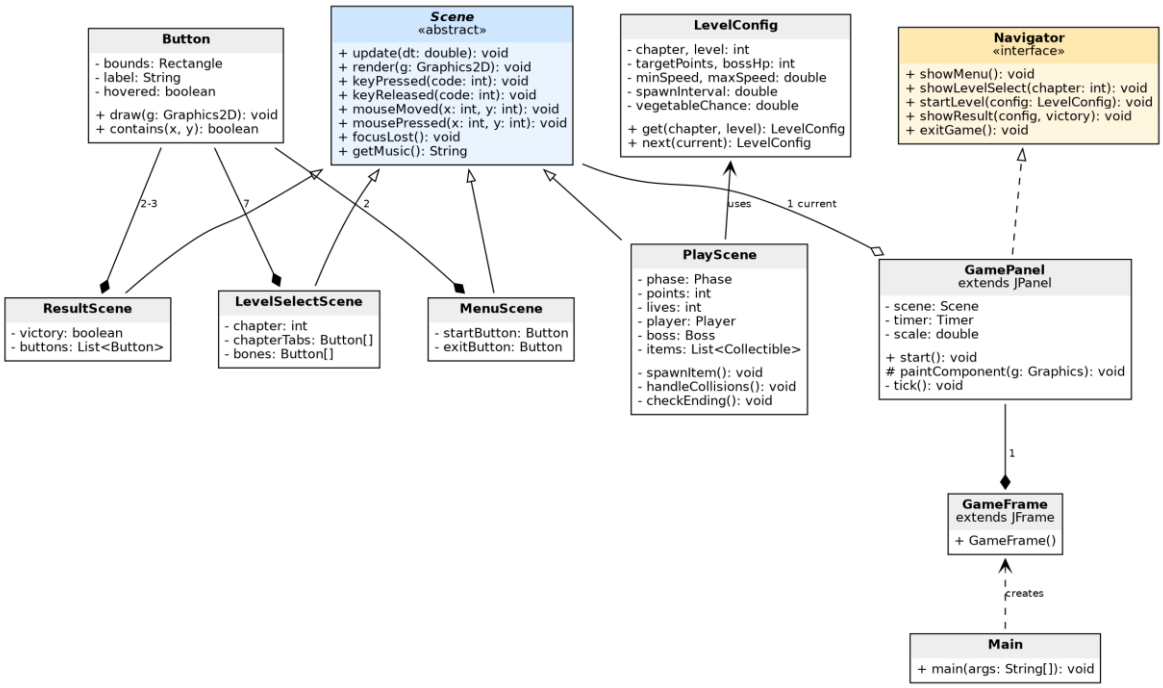
ระดับความยากของแต่ละด่าน

ความยากเพิ่มขึ้นตามลำดับด่านโดยปรับค่าต่างๆ ในคลาส LevelConfig ได้แก่ เป้าหมายแต้มหรือเลือดบอส ความเร็วของที่ตก ความถี่ที่ของเกิด สัดส่วนผัก และความเร็วของผู้เล่น

ด่าน	เป้าหมาย	ความเร็วของตก (พิกเซล/วินาที)	ของเกิดทุก (วินาที)	สัดส่วนผัก	ความเร็วผู้เล่น
1-1	150 แต้ม	250 - 320	0.90	14%	720
1-2	240 แต้ม	290 - 370	0.84	20%	720
1-3	บอส (เลือด 110)	300 - 380	0.78	20%	720
2-1	320 แต้ม	340 - 430	0.74	24%	780
2-2	400 แต้ม	370 - 470	0.68	27%	780
2-3	บอส (เลือด 170)	380 - 480	0.66	26%	780
3-1	480 แต้ม	420 - 520	0.62	30%	860
3-2	580 แต้ม	450 - 560	0.58	33%	860
3-3	บอส (เลือด 210)	430 - 530	0.62	22%	880

Class Diagram

ส่วนที่ 1 โครงสร้างหน้าจอและการควบคุมเกม



รูปที่ 2.1 คลาสไดอะแกรมของส่วนหน้าจอ (Scene) และการนำทาง

Scene – เป็น abstract class ของทุกหน้าจอในเกม กำหนดเมธอดมาตรฐาน เช่น update(), render(), keyPressed(), mousePressed() และ getMusic()

MenuScene, LevelSelectScene, PlayScene, ResultScene – สืบทอดจาก Scene หน้าจอเมนู หน้าเลือกบทและด่าน หน้าเล่นเกม และหน้าสรุปผล (ชนะ/แพ้)

Navigator – เป็น interface สำหรับสลับหน้าจอ ทำให้แต่ละ Scene สั่งเปลี่ยนหน้าได้โดยไม่ต้องรู้จัก GamePanel โดยตรง

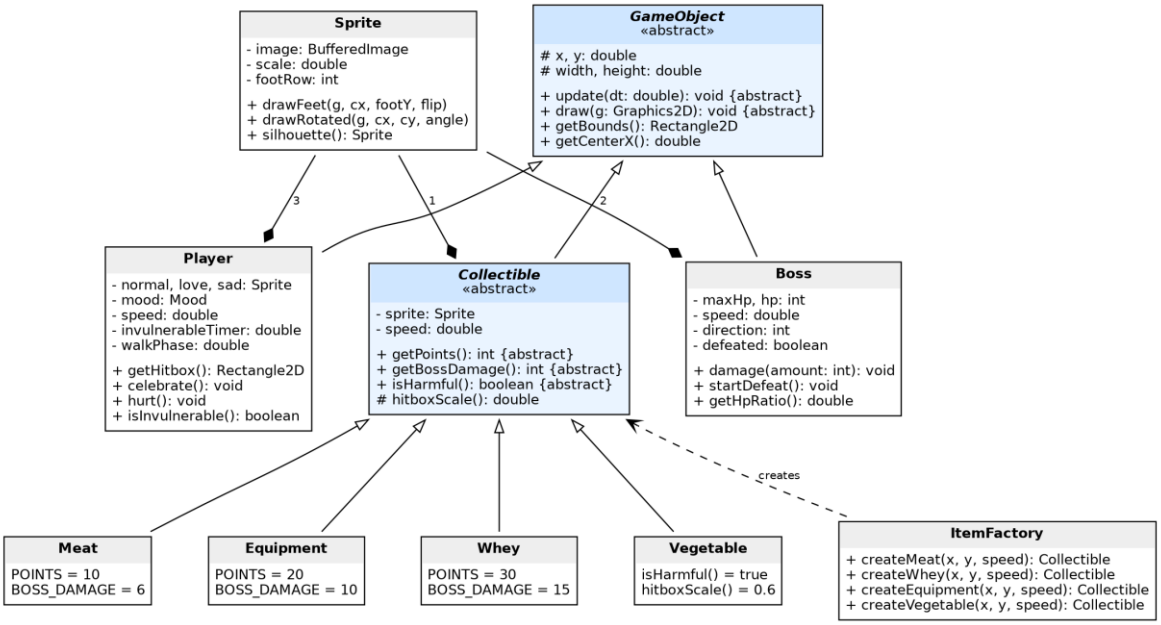
GamePanel – สืบทอดจาก JPanel และ implement Navigator ทำหน้าที่วนลูปเกมด้วย Timer วาดภาพ ปรับขนาดหน้าจอ รับ Event จากคีย์บอร์ดและเมาส์ ส่งต่อให้ Scene ปัจจุบัน

GameFrame, Main – GameFrame สืบทอดจาก JFrame สร้างหน้าต่างและใส่ GamePanel ส่วน Main เป็นคลาสเริ่มต้นของโปรแกรม

Button – ปุ่มที่วาดเอง มีทั้งแบบสีเขียวสไตล์เกมและแบบโปร่งใสที่วางทับปุ่มในภาพพื้นหลัง

LevelConfig – เก็บค่ากำหนดของทั้ง 9 ด่าน และมีเมธอด next() สำหรับหาด่านถัดไป

ส่วนที่ 2 วัตถุในเกม



รูปที่ 2.2 คลาสไดอะแกรมของวัตถุในเกม (GameObject)

GameObject – เป็น abstract class แม่ของวัตถุที่ขยับได้ทุกชนิด เก็บตำแหน่งและขนาด และมี getBounds() สำหรับตรวจสอบการชน

Player – หิวซอรัส เก็บภาพ 3 อารมณ์ (ปกติ, LOVE, SAD) ความเร็ว สถานะอมตะชั่วคราว และ Hitbox ที่เปลี่ยนตามบท

Boss – บอสประจำบท มีเลือด เดินไปมา กะพริบเมื่อโดนโจมตี และเฟดหายไปเมื่อแพ้

Collectible – เป็น abstract class ของของที่ตกลงมา กำหนดเมธอดนามธรรม getPoints(), getBossDamage() และ isHarmful()

Meat, Equipment, Whey, Vegetable – คลาสลูกของ Collectible แต่ละชนิดกำหนดค่าแต้ม ค่าลดเลือดบอส และความอันตรายต่างกัน

ItemFactory – สร้างของแต่ละชนิดโดยสุ่มรูปในกลุ่ม พร้อมจัดขนาดภาพให้เหมาะสม

Sprite – ห่อหุ้มภาพและขนาด มีเมธอดวาดภาพโดยอ้างอิงตำแหน่งเท้า วาดกลับซ้ายขวา และวาดหมุนได้

คลาสสนับสนุนอื่นๆ

คลาส	หน้าที่
Const	เก็บค่าคงที่ เช่น ขนาดหน้าจอ 1920x1080 ตำแหน่งพื้นที่เล่น และจำนวนชีวิต
Assets	โหลดและเก็บภาพ/ไฟล์เสียงไว้ในแคช โหลดได้ทั้งจากไฟล์ JAR และโฟลเดอร์ assets
Sounds	เล่นเสียงประกอบสั้นๆ (เก็บของกินได้ เก็บผัก บอสแพ้)
Music	เล่นเพลงพื้นหลังแบบวนซ้ำ แยกเพลงเมนู เพลงด่านปกติ และเพลงด่านบอส
UI	ฟังก์ชันช่วยวาดข้อความ เช่น จัดกึ่งกลาง ใส่ขอบ และย่อขนาดตัวอักษรให้พอดีกรอบ
FloatingText	ข้อความลอยขึ้นและค่อยๆ จางหาย เช่น +10 และ -1

รูปแบบการพัฒนา

- โปรแกรม: Visual Studio Code
- ภาษา: Java
- GUI: javax.swing (JFrame, JPanel) และ Java 2D (Graphics2D) สำหรับวาดภาพเกมทั้งหมด
- ระบบเสียง: javax.sound.sampled ทั้งเสียงประกอบและเพลงพื้นหลัง
- ภาพและตัวละคร: ไฟล์ภาพ PNG/JPG ที่ผู้จัดทำเตรียมไว้ในโฟลเดอร์ assets

แนวคิดการเขียนโปรแกรมเชิงวัตถุ

ตัวอย่างบางส่วนของโค้ดในเกมที่ใช้แนวคิดการเขียนโปรแกรมเชิงวัตถุ

- Constructor

```
public Player(int chapter, double speed, double fieldWidth, double fieldHeight) {
    super(0, 0, 0, 0);
    int index = chapter - 1;
    this.normal = new Sprite(Assets.image("player/MC" + chapter + ".png"), BODY_SCALE[index]);
    this.love = new Sprite(Assets.image("player/LOVE" + chapter + ".png"), LOVE_SCALE[index]);
    this.sad = new Sprite(Assets.image("player/SAD" + chapter + ".png"), SAD_SCALE[index]);
    this.speed = speed;
```

```
        this.fieldWidth = fieldWidth;
        this.hitboxWidth = HITBOX_WIDTH[index];
        this.width = normal.getWidth();
        this.height = normal.getHeight();
        this.x = (fieldWidth - width) / 2.0;
        this.y = fieldHeight - height - 6;
    }
```

1. Player() เป็น constructor ของคลาส Player รับค่าบท (chapter) ความเร็ว และขนาดพื้นที่เล่น
2. เรียก super(0, 0, 0, 0) เพื่อเรียก constructor ของคลาสแม่ GameObject ก่อน แล้วจึงตั้งค่าตัวแปรของตนเอง
3. เลือกโหลดภาพ MC / LOVE / SAD ตามหมายเลขบท จึงได้ตัวละครที่ถูกต้องตั้งแต่สร้างวัตถุ และวางตำแหน่งเริ่มต้นกลางหน้าจออัตโนมัติ

```
    public Boss(int chapter, int maxHp, double speed, double fieldWidth) {
        super(0, 70, 0, 0);
        this.sprite = new Sprite(Assets.image("boss/boss" + chapter + ".png"), DISPLAY_SCALE[chapter - 1]);
        this.flash = sprite.silhouette();
        this.maxHp = maxHp;
        this.hp = maxHp;
        this.lagHp = maxHp;
        this.speed = speed;
        this.fieldWidth = fieldWidth;
        this.width = sprite.getWidth();
        this.height = sprite.getHeight();
        this.x = (fieldWidth - width) / 2.0;
        this.turnTimer = 1.2 + random.nextDouble() * 1.5;
    }
```

constructor ของ Boss รับหมายเลขบท เลือดสูงสุด และความเร็ว เพื่อสร้างบอสประจำบทนั้น พร้อมสร้างภาพเงาสีขาวไว้ใช้กะพริบเมื่อโดนโจมตี

- Encapsulation

```
    private final Sprite normal;
    private final Sprite love;
    private final Sprite sad;
    private final double speed;
    private final double fieldWidth;
    private final double hitboxWidth;
    private Mood mood = Mood.NORMAL;
    private double moodTimer;
    private double invulnerableTimer;
```

```
private boolean movingLeft;
private boolean movingRight;
private boolean facingRight = true;
private boolean walking;
private double walkPhase;
private double idleTime;

public void celebrate() {
    if (mood != Mood.SAD) {
        mood = Mood.LOVE;
        moodTimer = 0.55;
    }
}

public void hurt() {
    mood = Mood.SAD;
    moodTimer = 0.9;
    invulnerableTimer = 1.4;
}

public void setFinalMood(Mood finalMood) {
    mood = finalMood;
    moodTimer = 1000;
}

public boolean isInvulnerable() {
    return invulnerableTimer > 0;
}
```

1. ตัวแปรของ Player เช่น normal, mood, speed, invulnerableTimer ถูกประกาศเป็น private ไม่สามารถแก้ไขจากภายนอกโดยตรง
2. ภายนอกเปลี่ยนสถานะผ่านเมธอดเท่านั้น เช่น celebrate() และ hurt() ซึ่งกำหนดอารมณ์และเวลาอมตะไว้ในที่เดียว
3. อ่านค่าได้ด้วย isInvulnerable() และ getMood() ทำให้ควบคุมกฎของเกมได้จากจุดเดียวและลดโอกาสเกิดข้อผิดพลาด

- Composition

```
private final Navigator navigator;
private final LevelConfig config;
private final Player player;
private final Boss boss;
private final List<Collectible> items = new ArrayList<>();
private final List<FloatingText> fieldTexts = new ArrayList<>();
```

```
    private final List<FloatingText> hudTexts = new ArrayList<>();
    private final Random random = new Random();
    private final BufferedImage background;
    private final BufferedImage leftPanel;
    private final BufferedImage heartImage;
    private final Button resumeButton;
    private final Button retryButton;
    private final Button menuButton;
```

1. PlayScene ประกอบด้วยวัตถุหลายชนิด ได้แก่ Player, Boss, รายการ Collectible, FloatingText, Button และภาพพื้นหลัง
2. LevelSelectScene ประกอบด้วย Button หลายอัน (แท็บบท กระดูกเลือกด่าน และปุ่มกลับ) ส่วน MenuScene และ ResultScene ก็มี Button เป็นส่วนประกอบเช่นกัน
3. Player และ Boss ประกอบด้วย Sprite หลายตัวสำหรับท่าทางต่างๆ จึงแยกหน้าที่การวาดภาพออกจากตรรกะของเกมได้ชัดเจน

- Polymorphism

```
public abstract class Collectible extends GameObject {
    private final Sprite sprite;
    private final double speed;
    private final double phase;
    private double age;

    protected Collectible(Sprite sprite, double centerX, double startY, double speed) {
        super(centerX - sprite.getWidth() / 2.0, startY, sprite.getWidth(), sprite.getHeight());
        this.sprite = sprite;
        this.speed = speed;
        this.phase = Math.random() * Math.PI * 2;
    }

    public abstract int getPoints();

    public abstract int getBossDamage();

    public abstract boolean isHarmful();

public class Vegetable extends Collectible {
    public Vegetable(Sprite sprite, double centerX, double startY, double speed) {
        super(sprite, centerX, startY, speed);
    }
```

```
    @Override
    protected double hitboxScale() {
        return 0.6;
    }

    @Override
    public int getPoints() {
        return 0;
    }

    @Override
    public int getBossDamage() {
        return 0;
    }

    @Override
    public boolean isHarmful() {
        return true;
    }
}
    private void handleCollisions() {
        Rectangle2D hitbox = player.getHitbox();
        Iterator<Collectible> it = items.iterator();
        while (it.hasNext()) {
            Collectible item = it.next();
            if (!item.getBounds().intersects(hitbox)) {
                continue;
            }
            if (item.isHarmful()) {
                if (player.isInvulnerable()) {
                    continue;
                }
                Sounds.play(Sounds.BAD);
                loseLife();
                it.remove();
            } else {
                collect(item);
                it.remove();
            }
```

```
        }
    }
```

1. ของทุกชนิดถูกเก็บไว้ในรายการเดียวคือ List<Collectible> และ PlayScene เรียก getPoints(), getBossDamage(), isHarmful() และ draw() โดยไม่ต้องรู้ว่าเป็นของชนิดใด
2. Meat, Equipment, Whey และ Vegetable override เมธอดเหล่านี้ให้ได้ค่าต่างกัน เช่น Whey ให้ 30 แต้ม ส่วน Vegetable ให้ isHarmful() เป็น true
3. Vegetable ยัง override hitboxScale() ให้ Hitbox เล็กกว่าของอื่น (0.6) เพื่อให้หลบง่ายขึ้น ส่วนคลาสอื่นใช้ค่าเริ่มต้น (0.8) การเพิ่มของชนิดใหม่จึงทำได้โดยไม่ต้องแก้โค้ดส่วนตรวจสอบการชน
4. ทุก Scene ก็ใช้หลักการเดียวกัน GamePanel เรียก update() และ render() ของ Scene ปัจจุบันโดยไม่ต้องรู้ว่าเป็นหน้าจอใด

- Abstract

```
public abstract class Scene {
    public abstract void update(double dt);

    public abstract void render(Graphics2D g);

    public void keyPressed(int keyCode) {
    }

    public void keyReleased(int keyCode) {
    }

    public void mouseMoved(int x, int y) {
    }

    public void mousePressed(int x, int y) {
    }

    public void focusLost() {
    }

    public String getMusic() {
        return null;
    }
}

public abstract class GameObject {
    protected double x;
```

```
protected double y;
protected double width;
protected double height;

protected GameObject(double x, double y, double width, double height) {
    this.x = x;
    this.y = y;
    this.width = width;
    this.height = height;
}

public abstract void update(double dt);

public abstract void draw(Graphics2D g);
```

1. Scene, GameObject และ Collectible เป็น abstract class ที่ไม่สามารถสร้างวัตถุโดยตรงได้
2. Scene บังคับให้คลาสลูกต้อง implement update() และ render() ส่วนเมธอดอื่น เช่น keyPressed() และ getMusic() มีค่าเริ่มต้นให้ แล้วคลาสลูกเลือก override เฉพาะที่ต้องใช้
3. GameObject บังคับให้ทุกวัตถุในเกมต้องมี update() และ draw() ของตนเอง ส่วน Collectible บังคับให้ของทุกชนิดต้องกำหนดแต้ม ค่าลดเลือดบอส และความอันตราย
4. ทำให้เพิ่มหน้าจอหรือของชนิดใหม่ได้โดยใช้โครงสร้างเดียวกัน

- Inheritance

```
public class Player extends GameObject
public class Boss extends GameObject
public abstract class Collectible extends GameObject
public class Meat extends Collectible
public class Vegetable extends Collectible
public class PlayScene extends Scene
public class GamePanel extends JPanel implements Navigator
```

1. Player, Boss และ Collectible สืบทอดจาก GameObject จึงใช้ตำแหน่ง ขนาด และ getBounds() ร่วมกัน ไม่ต้องเขียนซ้ำ
2. Meat, Equipment, Whey และ Vegetable สืบทอดจาก Collectible โดยเพิ่มเฉพาะค่าที่ต่างกัน
3. MenuScene, LevelSelectScene, PlayScene และ ResultScene สืบทอดจาก Scene
4. GamePanel สืบทอดจาก JPanel ของ Swing และ implement interface Navigator เพื่อใช้ความสามารถของ Swing ได้เต็มที่และเพิ่มระบบสลับหน้าจอของเกม

โครงสร้างGUI และ Component

เกมทั้งหมดแสดงผลบน JFrame เพียงหน้าต่างเดียว (GameFrame) ภายในมี JPanel หนึ่งตัว (GamePanel) ที่วาดทุกอย่างด้วย Graphics2D ในเมธอด paintComponent() โดยกำหนดพื้นที่ภาพภายในขนาด 1920x1080 แล้วย่อขยายให้พอดีกับหน้าต่างโดยรักษาสัดส่วน จึงปรับขนาดหน้าต่างได้โดยภาพไม่เพี้ยน ปุ่มต่างๆ เป็นคลาส Button

Event Handling

1. Event จากคีย์บอร์ดและเมาส์

```
timer = new Timer(15, e -> tick());
addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent e) {
        scene.keyPressed(e.getKeyCode());
    }

    @Override
    public void keyReleased(KeyEvent e) {
        scene.keyReleased(e.getKeyCode());
    }
});
MouseAdapter mouse = new MouseAdapter() {
    @Override
    public void mouseMoved(MouseEvent e) {
        scene.mouseMoved(toLogicalX(e), toLogicalY(e));
    }

    @Override
    public void mousePressed(MouseEvent e) {
        requestFocusInWindow();
        scene.mousePressed(toLogicalX(e), toLogicalY(e));
    }
};
addMouseListener(mouse);
addMouseMotionListener(mouse);
addFocusListener(new FocusAdapter() {
    @Override
    public void focusLost(FocusEvent e) {
```

```
            scene.focusLost();
        }
    });
```

- GamePanel ลงทะเบียน KeyListener, MouseListener, MouseMotionListener และ FocusListener แล้วส่ง Event ต่อให้ Scene ปัจจุบัน แต่ละหน้าจอตัดสินใจเองว่าจะทำอะไร
- ใช้ฟังก์ชันแปลงพิกัดเมาส์จากขนาดหน้าต่างจริงเป็นพิกัดภาพ 1920x1080 เพื่อให้คลิกปุ่มได้ตรงเสมอแม้ปรับขนาดหน้าต่าง

```
private int toLogicalX(MouseEvent e) {
    return (int) ((e.getX() - offsetX) / scale);
}

private int toLogicalY(MouseEvent e) {
    return (int) ((e.getY() - offsetY) / scale);
}
```

2. Event ในหน้าเล่นเกม

```
@Override
public void keyPressed(int keyCode) {
    if (keyCode == KeyEvent.VK_ESCAPE || keyCode == KeyEvent.VK_P) {
        togglePause();
        return;
    }
    if (phase == Phase.PAUSED) {
        if (keyCode == KeyEvent.VK_ENTER) {
            togglePause();
        } else if (keyCode == KeyEvent.VK_R) {
            navigator.startLevel(config);
        } else if (keyCode == KeyEvent.VK_M) {
            navigator.showMenu();
        }
        return;
    }
    if (keyCode == KeyEvent.VK_LEFT || keyCode == KeyEvent.VK_A) {
        player.setMovingLeft(true);
    } else if (keyCode == KeyEvent.VK_RIGHT || keyCode == KeyEvent.VK_D) {
        player.setMovingRight(true);
    } else if (phase == Phase.INTRO && (keyCode == KeyEvent.VK_ENTER || keyCode == KeyEvent.VK_SPACE)) {
        phaseTimer = 0;
    }
}
```

- กดปุ่มลูกศรซ้ายหรือ A และขวาหรือ D เพื่อตั้งค่าทิศทางการเคลื่อนที่ ปล่อยปุ่มเพื่อหยุด
- กด ESC หรือ P เพื่อหยุดชั่วคราว และกด ENTER หรือ SPACE ที่หน้าแนะนำด่านเพื่อข้ามไปเริ่มเล่น
- เมื่อหน้าต่างเสียโฟกัส (FocusListener) เกมจะหยุดชั่วคราวอัตโนมัติและหยุดการเคลื่อนที่ เพื่อไม่ให้ตัวละครวิ่งค้าง

3. Event เมนูและหน้าเลือกด่าน

- คลิก START (หรือกด ENTER) เพื่อเข้าหน้าเลือกด่าน คลิก EXIT เพื่อปิดโปรแกรม (System.exit(0))
- คลิกแท็บบทเพื่อเปลี่ยนบท คลิกกระดูกเพื่อเริ่มด่าน หรือกดปุ่ม 1 ถึง 3 เพื่อเริ่มด่านที่ตรงกัน กด ESC เพื่อกลับ

4. Event จากตัวจับเวลา (Timer)

```
private void tick() {
    long now = System.nanoTime();
    double dt = Math.min(0.05, (now - lastTime) / 1_000_000_000.0);
    lastTime = now;
    scene.update(dt);
    repaint();
    Toolkit.getDefaultToolkit().sync();
}
```

- javax.swing.Timer เรียก tick() ทุกประมาณ 15 มิลลิวินาที เพื่ออัปเดตเกมและสั่งวาดใหม่ แบบเรียลไทม์

5. Event ในหน้าสรุปผล

- คลิก NEXT LEVEL เพื่อไปด่านถัดไป RETRY เพื่อเล่นด่านเดิมใหม่ MENU เพื่อกลับเมนู

อัลกอริทึมสำคัญในโปรแกรม

1. ลูปเกมแบบใช้เวลาจริง (Delta Time)

คำนวณเวลาที่ผ่านไประหว่างเฟรม (dt) แล้วนำไปคูณกับความเร็วทุกอย่าง ทำให้เกมเร็วเท่ากันบนเครื่องที่ต่างกัน

2. การสุ่มสร้างของที่ตกลงมา (Spawn)

```
    double minGap = player.getHitbox().getWidth() + 150;
    for (int attempt = 0; attempt < 8 && Math.abs(x - lastSpawnX) < minGap; attempt++) {
        x = margin + random.nextDouble() * (Const.FIELD_W - margin * 2);
        if (boss != null) {
            x = boss.getCenterX() + (random.nextDouble() - 0.5) * boss.getWidth() * 0.9;
        }
    }
    x = Math.max(margin, Math.min(Const.FIELD_W - margin, x));
    lastSpawnX = x;
    double startY = boss != null ? boss.getMouthY() : -90;
```

```
        double speed = config.getMinSpeed() + random.nextDouble() * (config.getMaxSpeed() -
config.getMinSpeed());
        double vegetableChance = lastWasVegetable ? config.getVegetableChance() * 0.35 :
config.getVegetableChance();
        Collectible item;
        if (random.nextDouble() < vegetableChance) {
            item = ItemFactory.createVegetable(x, startY, speed);
            lastWasVegetable = true;
        } else {
            double roll = random.nextDouble();
            if (roll < config.getWheyChance()) {
                item = ItemFactory.createWhey(x, startY, speed);
            } else if (roll < config.getWheyChance() + config.getEquipmentChance()) {
                item = ItemFactory.createEquipment(x, startY, speed);
            } else {
                item = ItemFactory.createMeat(x, startY, speed);
            }
            lastWasVegetable = false;
        }
        items.add(item);
    }
```

- สุ่มตำแหน่งแนวนอน แต่ต้องห่างจากของชิ้นก่อนหน้าอย่างน้อย (ความกว้าง Hitbox ผู้เล่น + 150) เพื่อให้ผู้เล่นหลบผักได้
- สุ่มชนิดของตามสัดส่วนของด่าน หากชิ้นก่อนหน้าเป็นผักจะลดโอกาสเกิดผักซ้ำเหลือ 35% เพื่อไม่ให้ผักมารวมกันจนหลบไม่ได้
- ด่านบอสจะปล่อยของจากตำแหน่งปากบอส ผู้เล่นจึงต้องตามบอสไป และเมื่อเลือดบอสต่ำกว่า 50% ของจะเกิดถี่ขึ้นเล็กน้อย

3. การตรวจสอบการชน (Collision Detection) และ Hitbox

```
    public Rectangle2D getHitbox() {
        double w = width * hitboxWidth;
        return new Rectangle2D.Double(x + (width - w) / 2.0, y + height * 0.12, w, height * 0.78);
    }
```

- ใช้การชนแบบสี่เหลี่ยม (AABB) ด้วย Rectangle2D.intersects() ระหว่าง Hitbox ของผู้เล่นกับ Hitbox ของของแต่ละชิ้น
- Hitbox ของผู้เล่นเปลี่ยนตามขนาดตัวและตามบท (สัดส่วนความกว้าง 60%, 55% และ 45% ของภาพ)
  เป็นไปตามที่กำหนดไว้ใน Proposal ว่าตัวใหญ่ขึ้น Hitbox ก็เปลี่ยนตาม
- หลังโดนผัก ผู้เล่นจะอมตะ 1.4 วินาที (ภาพกะพริบ) เพื่อไม่ให้เสียชีวิตรัวจนเล่นไม่ไหว

4. การตัดสินแพ้ชนะและเลือดบอส

```
private void checkEnding() {
    if (lives <= 0) {
        endLevel(false);
    } else if (boss != null && boss.getHp() <= 0) {
        boss.startDefeat();
        Sounds.play(Sounds.BOSS_DIE);
        endLevel(true);
    } else if (boss == null && points >= config.getTargetPoints()) {
        endLevel(true);
    }
}

private void endLevel(boolean win) {
    phase = win ? Phase.WIN : Phase.LOSE;
    phaseTimer = win ? 1.9 : 1.6;
    items.clear();
    player.stopMoving();
    player.setFinalMood(win ? Player.Mood.LOVE : Player.Mood.SAD);
}
```

- แพ้เมื่อชีวิตเหลือ 0 ชนะด่านปกติเมื่อแต้มถึงเป้าหมาย และชนะด่านบอสเมื่อเลือดบอสเหลือ 0
- เมื่อจบด่านจะเปลี่ยนเฟสเป็น WIN หรือ LOSE แสดงอารมณ์ตัวละครและให้ผู้เล่นเห็นผลประมาณ 1.5 ถึง 2 วินาที ก่อนไปหน้าสรุปผล

5. ตรรกะของบอส

```
    x += direction * speed * dt;
    if (x < 0) {
        x = 0;
        direction = 1;
    } else if (x > fieldWidth - width) {
        x = fieldWidth - width;
        direction = -1;
    }
    turnTimer -= dt;
    if (turnTimer <= 0) {
        direction = -direction;
        turnTimer = 1.2 + random.nextDouble() * 1.8;
```

```
    }
}
```

- บอสเดินไปมาในแนวนอน เมื่อชนขอบจะกลับทิศ และสุ่มกลับทิศเองทุก 1.2 ถึง 3 วินาทีเพื่อให้คาดเดายาก
- แถบเลือดมีสองชั้น ชั้นสีแดงลดทันที ชั้นสีเหลืองค่อยๆ ลดตาม เพื่อให้เห็นผลของการโจมตี

6. อนิเมชันตัวละครและบอส

```
    double centerX = x + width / 2.0;
    double footY = y + height;
    double lift = 0;
    double tilt = 0;
    double scaleX = 1;
    double scaleY = 1 + Math.sin(idleTime * 3) * 0.012;
    if (walking) {
        double step = Math.abs(Math.sin(walkPhase));
        lift = step * height * 0.045;
        tilt = (facingRight ? 1 : -1) * 0.04 + Math.sin(walkPhase) * 0.05;
        scaleX = 1 + (1 - step) * 0.025;
        scaleY = 1 - (1 - step) * 0.035;
    }
    g.translate(centerX, footY - lift);
    g.rotate(tilt);
    g.scale(scaleX, scaleY);
    g.translate(-centerX, -footY);
```

- มีรูปตัวละครท่าเดียว จึงสร้างท่าเดินด้วยการแปลงภาพ (Transform) ตามจังหวะก้าว ได้แก่ เด้งขึ้นลง เอนตัว และยืดหดเล็กน้อย บอสใช้หลักการเดียวกัน และกลับซ้ายขวาตามทิศทางที่เดิน
- เมื่อเก็บของกินได้จะเปลี่ยนเป็นภาพ LOVE ชั่วครู่ และเมื่อเก็บผักจะเปลี่ยนเป็นภาพ SAD แล้วกลับมาเป็น MC เมื่อหมดเวลา

7. ระบบเสียง

```
@Override
public String getMusic() {
    return config.isBoss() ? Music.BOSS : Music.PLAY;
}

private void switchTo(Scene next) {
    scene = next;
    Music.play(next.getMusic());
    lastTime = System.nanoTime();
}
```

- แต่ละ Scene บอกชื่อเพลงของตนผ่าน getMusic() เมื่อสลับหน้าจอ GamePanel จะสั่งเล่นเพลงที่ตรงกัน หากเป็นเพลงเดิมก็จะเล่นต่อโดยไม่เริ่มใหม่
- เพลงพื้นหลังมี 3 เพลง (เมนู ด่านปกติ ด่านบอส) เล่นวนซ้ำ และเสียงประกอบ 3 เสียง (เก็บของกินได้ เก็บผัก และบอสแพ้) หากเครื่องไม่มีอุปกรณ์เสียง เกมจะปิดเสียงเองและเล่นต่อได้

บทที่ 3

สรุป

ปัญหาที่พบระหว่างการพัฒนา

1. ภาพตัวละครแต่ละท่า มีขนาดและสัดส่วนไม่เท่ากัน ทำให้ตัวละครกระโดดเมื่อสลับภาพ
   แก้โดยกำหนดสเกลแยกตามบทและวางภาพโดยยึดตำแหน่งเท้าเป็นหลัก
2. ตัวละครบทที่ 3 ตัวใหญ่มาก ทำให้หลบผักยากเกินไป แก้โดยปรับสัดส่วน Hitbox ตามบท ลด Hitbox ของผัก
   เพิ่มระยะห่างขั้นต่ำของของที่เกิด และให้อมตะชั่วคราวหลังโดนผัก
3. การปรับสมดุลความยาก ต้องไม่ง่ายเกินไปและไม่ยากเกินไปจนไม่สามารถผ่านด่านได้

จุดเด่นของเกมที่ไม่เหมือนใคร

1. ตัวละครมีการเติบโตแข็งแกร่งขึ้นตามบท
2. มีด่านบอสท้ายแต่ละบท พร้อมแถบเลือดบอส โดยใช้การเก็บเนื้อโจมตีแทนการเก็บแต้ม
3. ความยากเพิ่มขึ้นตามลำดับด่าน
4. มีอนิเมชันการเดิน เสียงประกอบ และเพลงพื้นหลังที่เปลี่ยนตามหน้าจอ
5. ปรับขนาดหน้าต่างได้โดยภาพไม่เพี้ยน

ข้อเสนอแนะและแนวทางพัฒนาต่อ

- เพิ่มระบบบันทึกความคืบหน้าและล็อกด่านตามลำดับ
- เพิ่มของพิเศษ เช่น ไอเท็มเพิ่มชีวิตหรือเพิ่มความเร็ว และเพิ่มอนิเมชันภาพต่อเนื่อง (Sprite Sheet) ให้ตัวละคร
- เพิ่มระบบเก็บคะแนนสูงสุดและตารางอันดับ

คำแนะนำสำหรับผู้สอนที่อยากให้อธิบาย หรือที่เรียนแล้วไม่เข้าใจ หรืออยากให้เพิ่มสำหรับ น้อง ๆ รุ่นต่อไป

- ไม่มี